

SecureDrive — Security Threat Analysis

MITRE ATT&CK Threat Modelling + OWASP Top 10:2025 Compliance

Yashaswini L & Raghuv eer Verma · IIT Dharwad, CSE · BTP 2026

Part 1: MITRE ATT&CK Threat Analysis

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) is a globally recognised knowledge base of adversary tactics and techniques based on real-world observations. This section maps 13 relevant MITRE ATT&CK Enterprise techniques to SecureDrive's architecture and documents the mitigation status of each.

Status legend:

- ☒ Mitigated — control implemented and verified
- ☒ By Design — architecture makes the attack impossible regardless of other controls
- ☐ Partial — control exists but a gap remains with documented remediation plan
- ☐ Open — no control in place

Summary Table

ID	Technique	STRIDE	Status	Mitigation
T1078	Valid Accounts — login without password	Spoofing	☒ Mitigated	bcrypt verification + JWT (SD-V001)
T1110	Brute Force — password guessing	Spoofing	☒ Mitigated	Rate limiting + PBKDF2 600k iterations
T1552	Unsecured Credentials in source code	Info Disclosure	☒ Mitigated	.env + .gitignore + Render env vars
T1530	Data from Cloud Storage (S3 breach)	Info Disclosure	☒ Mitigated	S3 stores only AES-256-GCM ciphertext
T1557	Adversary-in-the-Middle	Tampering	☒ Mitigated	HTTPS enforced on Render + Netlify
T1565	Data Manipulation — ciphertext tampering	Tampering	☒ Mitigated	AES-GCM authentication tag
T1499	Endpoint DoS — API flooding	DoS	☒ Mitigated	express-rate-limit 100 req/15 min
T1190	Exploit Public App — unauthorised file access	Privilege Esc	☒ Mitigated	Ownership check on all 5 file routes
T1059	Script injection via filename	Execution	☒ Mitigated	Filename sanitisation + escapeHtml()
T1539	Steal Web Session Token via XSS	Spoofing	☒ Mitigated	CSP headers + non-extractable CryptoKey
T1005	Data from Local System — key in storage	Info Disclosure	☒ Mitigated	Master Key in JS module memory only
T1498	Network DDoS — volumetric attack	DoS	☐ Partial	Rate limiting active; Cloudflare planned on custom domain
T1036	Masquerading — malicious file upload	Defense Evasion	☒ By Design	Encryption makes server-side execution impossible

Detailed Analysis

T1078 — Valid Accounts

An attacker who knows a victim's email address could obtain a valid JWT token without knowing their password. This was identified as vulnerability SD-V001 during internal peer code review (Raghuveer Verma).

Fix: bcrypt password hash (cost factor 12) stored at signup. bcrypt.compare() verification at login before any token is issued. Generic error messages prevent user enumeration.

Status: ☒ Fully mitigated. Verified against all four attack scenarios.

T1110 — Brute Force

Systematic password guessing against the login endpoint.

Fix: express-rate-limit (100 requests/IP/15 min) at application level. PBKDF2 with 600,000 iterations means even with a stolen hash, each offline guess takes ~1 second. bcrypt cost 12 adds additional server-side cost per attempt.

Status: ☒ Fully mitigated.

T1552 — Unsecured Credentials

MongoDB URI, JWT secret, and AWS keys hardcoded in source code or committed to GitHub.

Fix: All secrets stored in .env (listed in .gitignore). Production secrets stored as Render environment variables — never in source code. Git history verified to contain no secrets.

Status: ☒ Fully mitigated.

T1530 — Data from Cloud Storage

Attacker gains access to S3 bucket and downloads stored files.

Fix: All files encrypted client-side with AES-256-GCM before upload. S3 receives only ciphertext — useless without the per-file CEK, which requires the Master Key, which requires the user's password. Even complete S3 compromise yields no plaintext.

Status: ☒ Mitigated by design.

T1557 — Adversary-in-the-Middle

Attacker intercepts traffic between client and server.

Fix: All client-server communication is protected using HTTPS/TLS (Render and Netlify). In addition, SecureDrive performs client-side end-to-end encryption before upload. Therefore, intercepted file contents remain encrypted using AES-256-GCM, and the server never possesses the plaintext encryption keys. An attacker intercepting network traffic or obtaining stored encrypted files cannot recover the original file contents without the user's cryptographic keys.

Status: ☒ Fully mitigated — TLS + client-side end-to-end encryption.

T1565 — Data Manipulation

Attacker modifies encrypted file bytes stored in S3.

Fix: AES-GCM produces a 128-bit authentication tag per file. Any modification to the ciphertext causes decryption to fail — the Web Crypto API throws an error before returning any bytes to the user.

Status: ☒ Cryptographic guarantee — cannot be bypassed.

T1499 — Endpoint Denial of Service

Attacker floods API with requests to exhaust server resources.

Fix: express-rate-limit middleware returns 429 Too Many Requests after 100 requests per IP per 15-minute window. Multer 50MB file size limit prevents resource exhaustion via large uploads.

Status: ☒ Application-level DoS mitigated.

T1190 — Exploit Public-Facing Application

Attacker with a valid JWT token accesses files belonging to other users by guessing MongoDB ObjectIDs.

Fix: Every file database query includes { _id: fileId, owner: req.user.userId }. A file belonging to another user returns null → 404. Applied on all five file routes.

Status: ☒ Fully mitigated.

T1059 — Script Injection via Filename

Attacker uploads a file with a malicious name containing HTML/script tags, path traversal sequences, or null bytes. When the filename is rendered in the dashboard, it executes as HTML.

Fix: Backend sanitizeFilename() strips non-alphanumeric characters and limits length to 255 chars. Frontend escapeHtml() encodes all special characters before rendering filenames in the DOM.

Status: ☒ Fully mitigated at both backend (storage) and frontend (rendering).

T1539 — Steal Web Session Token

XSS attack reads JWT token from storage and exfiltrates it.

Fix: Content-Security-Policy header restricts script execution to 'self' — injected scripts are blocked by the browser. X-Content-Type-Options, X-Frame-Options (DENY), and Referrer-Policy headers add additional protection layers.

Status: ☒ Mitigated via CSP headers and security header suite.

T1005 — Data from Local System (Master Key)

XSS attack reads Master Key from sessionStorage and uses it to decrypt user files.

Fix: Master Key stored as a non-extractable Web Crypto CryptoKey object in JavaScript module memory — never exported to bytes, never written to sessionStorage or localStorage. A non-extractable key cannot be read by any JavaScript, including injected XSS code. Key is cleared from memory on logout or tab close.

Status: ☒ Fully mitigated. Non-extractable key implementation complete.

T1498 — Network DDoS (Volumetric)

Large-scale distributed attack from thousands of IPs overwhelming the server at network level — beyond what application-level rate limiting can handle.

Current status: Application-level rate limiting (express-rate-limit) is active and mitigates per-IP flooding. The application is currently deployed on Render's free tier using the .onrender.com subdomain. Cloudflare-based network-level DDoS protection is planned for future deployment once a custom domain is acquired — Cloudflare requires DNS control which is only possible with a custom domain.

Status: ☐ Partial. Rate limiting active; Cloudflare DDoS protection planned as future enhancement.

T1036 — Masquerading / Malicious File Upload

Attacker uploads a malicious file (.sh, .exe, malware) disguised as a benign file.

Architectural decision: File type restriction is deliberately not implemented. SecureDrive's core value proposition is private storage for any file type — restricting file types would exclude legitimate users (developers backing up scripts, sysadmins storing configs). More importantly, all file content is encrypted client-side before reaching the server using AES-256-GCM. The server stores only encrypted bytes and never executes, interprets, or processes file content. A .sh file encrypted in S3 is cryptographically indistinguishable from a .pdf file — and neither can be executed by the server.

Status: ☒ Mitigated by design. Encryption makes server-side execution impossible regardless of file type.

Part 2: OWASP Top 10:2025 Compliance

The OWASP Top 10 is the industry-standard awareness document for web application security risks. This section maps SecureDrive's implementation against the OWASP Top 10:2025 list.

Compliance Summary Table

ID	Risk	Status	SecureDrive Mitigation
A01	Broken Access Control	✔ Mitigated	JWT auth + per-route ownership validation. Users can only access their own files.
A02	Security Misconfiguration	✔ Mitigated	CSP headers, X-Frame-Options, nosniff, Referrer-Policy all set. Secrets in env vars.
A03	Software Supply Chain Failures	✔ Mitigated	All dependencies pinned in package.json. No unvetted third-party scripts loaded.
A04	Cryptographic Failures	✔ Mitigated	AES-256-GCM, PBKDF2 600k iter (NIST 2023), bcrypt cost 12. No weak algorithms used.
A05	Injection	✔ Mitigated	Filename sanitisation, escapeHtml() on all rendered filenames. Mongoose prevents NoSQL injection.
A06	Insecure Design	✔ Mitigated	Three-layer key hierarchy designed from scratch. STRIDE + MITRE ATT&CK modelled upfront.
A07	Authentication Failures	✔ Mitigated	bcrypt password hash, JWT with expiry, SD-V001 fix. No default credentials.
A08	Software/Data Integrity Failures	✔ Mitigated	AES-GCM auth tag verifies file integrity on every download. Any tampering fails decryption.
A09	Security Logging & Alerting Failures	⚠ Partial	Basic server-side error logging via console.error. Structured audit log planned (blockchain trail).
A10	Mishandling of Exceptional Conditions	✔ Mitigated	Try-catch on all async routes. Generic error messages to client prevent info leakage.

Detailed Analysis

A01:2025 — Broken Access Control

Access control failures allow users to act outside of their intended permissions — accessing other users' data, viewing sensitive endpoints, or performing privileged actions.

SecureDrive mitigation: JWT authentication middleware protects all file endpoints. Every database query includes an ownership check (owner: req.user.userId) — a user can only list, download, or delete files they own. Unauthenticated requests receive 401 before reaching any business logic.

Status: ✔ Mitigated.

A02:2025 — Security Misconfiguration

Missing security headers, default credentials, unnecessary features enabled, or improperly configured cloud storage.

SecureDrive mitigation: Full security header suite implemented (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy). No default credentials anywhere in the system. All secrets in environment variables. S3 bucket does not allow public listing.

Status: ✔ Mitigated.

A03:2025 — Software Supply Chain Failures

Use of components with known vulnerabilities, unverified third-party packages, or tampered dependencies.

SecureDrive mitigation: All dependencies pinned to specific versions in package.json. No third-party CDN scripts loaded in the frontend — all crypto runs via the browser's built-in Web Crypto API with no external library. npm audit should be run periodically to check for known CVEs.

Status: ☒ Mitigated. Recommendation: add npm audit to deployment pipeline.

A04:2025 — Cryptographic Failures

Weak encryption algorithms, insufficient key lengths, improper key management, or secrets transmitted in plaintext.

SecureDrive mitigation: AES-256-GCM (authenticated encryption, unique IV per file). PBKDF2-SHA256 with 600,000 iterations — NIST SP 800-132 (2023) compliant. bcrypt cost factor 12 for password hashing. All keys derived client-side — server never receives plaintext keys or passwords. HTTPS enforced for all traffic.

Status: ☒ Fully mitigated. Cryptographic choices meet or exceed current NIST standards.

A05:2025 — Injection

SQL, NoSQL, or command injection through unsanitised user input.

SecureDrive mitigation: Mongoose ODM with typed schemas prevents NoSQL injection — queries use structured objects, not string interpolation. Filename sanitisation (sanitizeFilename()) strips dangerous characters before DB storage. escapeHtml() applied to all user-controlled strings before DOM rendering.

Status: ☒ Mitigated.

A06:2025 — Insecure Design

Missing or ineffective security controls by design — threat modelling not performed, security requirements not defined upfront.

SecureDrive mitigation: Three-layer key hierarchy (password → Master Key → per-file CEK) designed upfront with explicit security goals. STRIDE threat model completed before implementation. MITRE ATT&CK mapping performed post-implementation. NIST SSDF followed as development framework. Security requirements documented formally.

Status: ☒ Mitigated. Security was a design-time concern, not an afterthought.

A07:2025 — Authentication Failures

Weak passwords, credential stuffing, missing brute force protection, insecure session management.

SecureDrive mitigation: bcrypt password hashing (cost 12). Minimum password length enforced (8 characters). Rate limiting prevents credential stuffing. JWT with 7-day expiry. Session cleared on logout. SD-V001 fix ensures password verification before token issuance.

Status: ☒ Fully mitigated.

A08:2025 — Software and Data Integrity Failures

Code and infrastructure not protected against integrity violations — unsigned updates, tampered data, insecure CI/CD pipelines.

SecureDrive mitigation: AES-GCM authentication tag provides cryptographic integrity verification of every stored file — any modification is detected on download. GitHub is the single source of truth for code. Render deploys directly from GitHub with no manual file transfer.

Status: ☒ Mitigated. File integrity cryptographically guaranteed.

A09:2025 — Security Logging and Alerting Failures

Insufficient logging of security events — login failures, access control violations, and suspicious activity not recorded or alerted on.

SecureDrive current state: Basic error logging via console.error() on the backend. Login failures and unauthorised access attempts produce log entries visible in Render's dashboard. However, no structured security event log, no alerting system, and no centralised log aggregation is currently in place.

Planned mitigation: The blockchain-anchored audit trail (BTP novel contribution) will provide tamper-evident logging of all file access events. Server-side security event logging (failed logins, 401s, rate limit triggers) is planned as a future enhancement.

Status: ⚠ Partial. Basic logging active; structured security audit log planned via blockchain trail.

A10:2025 — Mishandling of Exceptional Conditions

Unhandled errors exposing stack traces, internal paths, or sensitive system information to the client.

SecureDrive mitigation: All async route handlers wrapped in try-catch. Error responses return generic messages ('Upload failed', 'Download failed') — no stack traces, file paths, or internal details sent to client. Server-side error details logged internally only.

Status: ☒ Fully mitigated.

Overall Security Posture Summary

Framework	☒ Mitigated	⚠ Partial	✖ Open
MITRE ATT&CK (13 techniques)	12 / 13	1 / 13 (T1498)	0 / 13
OWASP Top 10:2025 (10 risks)	9 / 10	1 / 10 (A09)	0 / 10

SecureDrive addresses 12 of 13 MITRE ATT&CK techniques and 9 of 10 OWASP Top 10:2025 risks. The single remaining partial mitigation in each framework — network-level DDoS (T1498) and structured security logging (A09) — are both planned enhancements. T1498 will be addressed via Cloudflare when a custom domain is acquired. A09 will be addressed by the blockchain-anchored audit trail, which is the project's primary novel contribution and the focus of the next BTP phase.

No open vulnerabilities remain in the current implementation. This represents a strong security posture for a prototype at TRL Level 4.
